
CS 559: FORECASTING BIKE AVAILABILITY AND STATION CONGESTION IN THE NYC CITI BIKE NETWORK

Milan Chandiramani*, Komal Wavhal*, Matthew Schmitt*

*Stevens Institute of Technology
Fall 2025

ABSTRACT

Accurate short-term forecasting of bike-sharing station conditions is critical for reducing congestion, improving service reliability, and enabling proactive bike rebalancing. However, most public bike-sharing datasets focus on trip-level demand and lack direct observations of station availability. In this project, we develop and compare three forecasting approaches to predict *available bikes*, *available docks*, and *station occupancy* at an hourly resolution across the NYC Citi Bike network.

Using a large-scale station-status dataset derived from Open Bus Project feeds (2015–2019), we evaluate a global multi-output XGBoost model, a global multi-output LSTM model, and ARIMA/ARIMAX baselines. Experimental results demonstrate that machine learning models substantially outperform classical time-series approaches, with XGBoost achieving the most accurate and stable forecasts across all targets.

1 Introduction

Bike-sharing systems play an important role in urban transportation by providing last-mile connectivity and reducing traffic congestion. A persistent challenge in such systems is *station congestion*, where stations become either empty (no bikes available) or full (no docks available). These conditions reduce rider satisfaction and increase operational costs due to reactive rebalancing.

Accurate short-term forecasting of station availability enables operators to anticipate congestion, optimize redistribution strategies, and improve system reliability. However, commonly used Citi Bike trip datasets contain only ride-level information and do not record station availability or occupancy.

Objective. This project aims to forecast station-level operational conditions using direct station-status data. Specifically, we predict:

- Available bikes
- Available docks
- Occupancy (ratio of bikes to total docks)

We formulate this task as a supervised time-series regression problem and compare tree-based, deep learning, and classical statistical methods.

2 Related Work

Early work on bike-sharing forecasting relied on statistical time-series models such as ARIMA and SARIMA [1], which capture trend and seasonality but assume linear dynamics. While interpretable, these models struggle with nonlinear demand patterns and exogenous influences.

More recent studies employ machine learning approaches such as Random Forests and Gradient Boosting, demonstrating improved accuracy for bike availability prediction [2]. Facebook Prophet [4] extends classical decomposition-based forecasting but remains limited in modeling complex interactions.

Deep learning models, particularly Long Short-Term Memory (LSTM) networks [3], have shown strong performance in transportation and mobility forecasting due to their ability to capture long-range temporal dependencies. Graph-based approaches further incorporate spatial relationships between stations [5], but require complex graph construction and high computational cost.

Most prior work focuses on predicting trip demand or pickup counts rather than directly forecasting operational congestion metrics such as available docks. This project addresses that gap using station-status data.

3 Methodology

We adopt a unified modeling approach in which all methods are trained on the same hourly-resampled feature set and evaluated on identical train–test splits. Each model receives identical temporal, spatial, lagged, and weather features as input and produces predictions for all three operational targets¹. This ensures that performance differences reflect modeling capacity rather than feature availability.

3.1 Problem Formulation

Let s denote a bike station and t an hourly time index. We define the target vector:

$$\mathbf{y}_{s,t} = \begin{bmatrix} \text{avail_bikes}_{s,t} \\ \text{avail_docks}_{s,t} \\ \text{occupancy}_{s,t} \end{bmatrix}$$

Given historical observations within a window of length w , the goal is to learn:

$$\hat{\mathbf{y}}_{s,t} = f(\mathbf{X}_{s,t-w:t-1})$$

where $\mathbf{X}$ contains temporal, spatial, lagged, and weather features.

3.2 XGBoost (Global Multi-Output Tree-Based Regression)

Extreme Gradient Boosting (XGBoost) is a tree-based ensemble learning method that models predictions as an additive combination of regression trees. Given an input feature vector $\mathbf{x}_i$, the prediction is expressed as:

$$\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i), \quad f_k \in \mathcal{F}$$

where $\mathcal{F}$ denotes the space of decision trees. Each tree incrementally corrects the residual errors of previous trees, enabling the model to capture nonlinear relationships and feature interactions.

Global Multi-Output Training. We train a **single global XGBoost model** across all stations rather than individual station-level models. Multi-output forecasting is implemented using a wrapper that trains one regressor per target variable while sharing the same engineered feature set. This approach allows the model to generalize across stations while maintaining independent error minimization for available bikes, available docks, and occupancy.

Objective Function. Model training minimizes a regularized empirical loss:

$$\mathcal{L} = \sum_i \ell(y_i, \hat{y}_i) + \sum_k \Omega(f_k),$$

where $\ell(\cdot)$ is the squared error loss and $\Omega(\cdot)$ penalizes tree complexity through constraints on depth and leaf weights [2]. This regularization helps prevent overfitting while preserving predictive accuracy.

Feature Handling and Hyperparameters. Unlike neural networks, XGBoost does not require explicit feature scaling and can naturally handle heterogeneous feature types. Based on empirical tuning, we use 400 boosting rounds with a learning rate of 0.05 and a maximum tree depth of 6. Row and column subsampling rates are set to 0.9 to reduce variance and improve generalization across stations. All models are trained with parallel computation to ensure scalability over millions of hourly observations.

¹Note: We make an exception when training prophet. Its methodology is detailed in its section

Model Behavior. The tree-based structure enables XGBoost to respond sharply to regime changes, such as sudden station emptying or saturation events, which are difficult for linear time-series models to capture. As a result, XGBoost serves as a strong nonlinear baseline and provides an effective comparison point for deep learning approaches.

3.3 LSTM (Global Multi-Output Sequence-to-One Regression)

Long Short-Term Memory (LSTM) networks model temporal dependencies using gated recurrent units that maintain an internal memory state across time:

$$\mathbf{h}_t = \text{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1})$$

To capture both short-term and daily temporal patterns, we adopt a *sequence-to-one* forecasting formulation. For each station, a sliding window of $w = 48$ hours is used as input, and the model predicts the operational state at the next hour. Formally, the final hidden state is mapped to the output vector as:

$$\hat{\mathbf{y}}_t = \mathbf{W}\mathbf{h}_t + \mathbf{b}$$

where $\hat{\mathbf{y}}_t$ jointly represents available bikes, available docks, and occupancy.

Global Multi-Output Training. Rather than training separate models per station, we train a **single global LSTM model** across all stations. This enables the model to learn shared temporal dynamics while generalizing across diverse station behaviors. All three targets are predicted simultaneously using a multi-output regression head.

Input Preprocessing. Since LSTMs are sensitive to feature scale, all input features and target variables are standardized using z-score normalization prior to training. The same scalers are applied during inference to ensure consistency between training and test data.

Architecture. The network consists of two stacked LSTM layers with dropout regularization, followed by fully connected layers:

$$\text{LSTM}(128) \rightarrow \text{Dropout}(0.2) \rightarrow \text{LSTM}(128) \rightarrow \text{Dropout}(0.2) \rightarrow \text{Dense}(128) \rightarrow \text{Dense}(3)$$

Training Configuration. The model is trained using the Adam optimizer with a learning rate of 3×10^{-4} and a batch size of 1024. A Huber loss function is used to balance sensitivity between small errors and robustness to large deviations caused by abrupt congestion events. Early stopping with a patience of five epochs is employed based on validation loss to prevent overfitting.

Rationale for Sequence-to-One Formulation. We adopt a sequence-to-one forecasting formulation because the objective of this study is to predict the *next-hour operational state* of a station based on recent historical context. This setup aligns naturally with short-term congestion forecasting, where operators require a single, actionable prediction at each time step rather than a full future trajectory.

Compared to sequence-to-sequence models, sequence-to-one regression reduces model complexity, limits error accumulation, and enables direct comparison with non-sequential models such as XGBoost and ARIMA. By conditioning predictions on a fixed-length historical window, the LSTM can effectively capture temporal dependencies while remaining computationally efficient and stable for large-scale hourly forecasting.

Implementation Details. Training is performed on GPU hardware with mixed-precision computation to accelerate optimization and reduce memory usage. During inference, the model produces one-step ahead predictions for each hour based on the preceding 48-hour context, enabling direct comparison with non-sequential models such as XGBoost.

Overall, this architecture enables the LSTM to capture nonlinear and long-range temporal dependencies in station availability while remaining scalable to millions of hourly observations [3].

3.4 ARIMA / ARIMAX (Global Statistical Baselines)

Autoregressive Integrated Moving Average (ARIMA) models represent a classical time-series forecasting approach that captures linear temporal dependencies through autoregressive and moving-average components. A univariate ARIMA(p, d, q) model is defined as:

$$\phi(B)(1 - B)^d y_t = \theta(B)\epsilon_t,$$

where B is the backshift operator, p and q denote the autoregressive and moving-average orders, d is the differencing order, and ϵ_t is white noise.

Seasonal ARIMA. To account for strong daily periodicity in bike-sharing usage, we employ a seasonal extension with a 24-hour cycle. The resulting SARIMA formulation captures both short-term autocorrelation and daily seasonality:

$$\text{SARIMA}(p, d, q) \times (P, D, Q)_{24}.$$

In our experiments, model orders are fixed to $(1, 0, 1) \times (1, 0, 1)_{24}$ based on empirical stability and computational tractability.

ARIMAX with Exogenous Variables. ARIMAX extends ARIMA by incorporating external regressors:

$$y_t = \beta^\top \mathbf{X}_t + \text{ARIMA}(p, d, q),$$

where $\mathbf{X}_t$ includes temporal indicators, lagged occupancy features, rolling averages, spatial attributes, and weather variables. This formulation allows the model to account for known covariates while preserving the interpretability of linear time-series dynamics.

Global Modeling Strategy. Rather than fitting separate models for each station, we train ARIMA and ARIMAX models on a **globally aggregated hourly time series**. This approach provides a computationally feasible statistical baseline and ensures consistency with the global modeling paradigm used for XGBoost and LSTM. Separate models are trained for each target variable: available bikes, available docks, and occupancy.

Training and Inference. Models are estimated using maximum likelihood with relaxed stationarity and invertibility constraints to improve convergence on large-scale data. One-step-ahead forecasts are generated over the test horizon and evaluated using the same metrics as machine learning models.

Limitations. Despite their interpretability, ARIMA-based models assume linear relationships and struggle to adapt to abrupt regime changes such as rapid station emptying or saturation events. As a result, they serve primarily as statistical baselines for quantifying the gains achieved by nonlinear machine learning and deep learning approaches [1, 6].

3.5 Prophet

To establish a baseline for “normal” Citi Bike station occupancy dynamics, we implemented a forecasting pipeline using Prophet, a decomposable time-series regression model that represents the observed series as the sum of a trend component and multiple seasonal and regressor-driven effects.

Data preparation and feature engineering We began from station-level observations containing a timestamp, station identifier (*dock_name*), occupancy, station capacity (*tot_docks*), and weather covariates. Timestamps were parsed into a Prophet-compatible datetime field (*ds*) and the data were resampled onto a fixed, regular time grid. For each station and each time bin, we retained the last observed state and forward-filled missing bins to obtain a complete, regularly sampled panel. We then aggregated across stations by summing occupancy over stations per time bin, and computed the mean of the available weather variables per bin (with interpolation to fill gaps). The resulting design matrix includes calendar features (e.g., *is_weekend*) and rush-hour indicators (*is_rush_am*, *is_rush_pm*) derived from local weekday/hour-of-day logic. Because station capacity varies over time and across stations, the target was normalized as an occupancy rate by dividing occupancy by total docks (with safeguards to avoid division by zero). In addition, we created autoregressive-style lag features for the normalized target (ex: 24-hour and 7-day lags) to provide Prophet with short-term persistence signals alongside the seasonal structure. These engineered features were treated as external regressors during model fitting, consistent with Prophet’s requirement that any regressor columns be present in both the historical (fit) dataframe and the prediction (future) dataframe.

Model specification We configured Prophet to learn a flexible baseline with an automatically selected piecewise trend with a limited number of changepoints, and multiple seasonal components. In addition to weekly seasonality, we explicitly added a custom within-day seasonality (“*daily_base*”) using Fourier terms. We also modeled rush-hour structure by adding conditional daily seasonalities that only activate when the corresponding binary indicator is true (AM vs. PM rush periods), enabling different intraday shapes for commute windows versus non-commute periods (an approach aligned with Prophet’s conditional seasonality mechanism). Finally, we added weather and lagged-occupancy covariates as extra regressors via *add_regressor*, allowing these features to enter the linear portion of the model.

Hyperparameter tuning via time-series cross-validation To select an appropriate bias–variance tradeoff for trend and seasonality flexibility, we performed a grid search over key Prophet hyperparameters, including *changepoint_prior_scale*, *seasonality_prior_scale*, and *seasonality_mode* (additive vs. multiplicative). Model configurations were evaluated using Prophet’s rolling-origin time-series cross-validation, where the model is repeatedly refit on data up to a cutoff date and evaluated over a forward forecast horizon; the procedure is controlled by the forecast horizon (horizon), cutoff spacing (period), and the size of the initial training window (initial). For each configuration, we computed standard forecast error metrics from the cross-validation predictions (ex: RMSE, MAE, MAPE) and selected the best-performing setting based on aggregate error.

Holdout evaluation and error inspection After choosing the best hyperparameters, we refit Prophet on the full training portion of the time series and generated forecasts on a temporally held-out test interval. We reported holdout MAE and also expressed MAE relative to the average test-set occupancy rate to contextualize the magnitude of typical errors. To understand failure modes, we joined predictions back to the feature rows and inspected the largest absolute-error cases.

3.6 Model Limitations

While the evaluated models provide strong forecasting performance, each approach exhibits inherent limitations that are important to acknowledge.

XGBoost relies on fixed-length historical features and does not explicitly model long-range temporal dependencies beyond the engineered lag and rolling-window features. Although it performs well for short-term forecasting, extending predictions to longer horizons would require iterative or multi-step modeling strategies.

LSTM models capture nonlinear and long-term temporal patterns but are more computationally expensive to train and sensitive to hyperparameter choices such as window length and learning rate. In addition, sequence-to-one formulations may smooth sharp regime changes, leading to higher error during abrupt congestion events.

ARIMA and ARIMAX assume linear relationships and stationarity, which limits their ability to adapt to complex, nonlinear dynamics present in urban mobility systems. While interpretable, these models struggle to capture sudden shifts in station usage and therefore serve primarily as statistical baselines.

Prophet is a decomposable nonlinear regression that assumes the remaining error is white noise and works best when the series has stable seasonality over multiple cycles. For our station-occupancy data, sharp step changes and operational shocks often leave substantial residual autocorrelation that Prophet’s smooth components do not capture well, so it can yield higher MAE than ARIMA, LSTM, and XGBoost.

Addressing these limitations—particularly spatial dependencies and multi-step forecasting—remains an important direction for future work.

3.7 Feature Engineering

All models operate on an hourly-resampled global dataset with the following features:

- Temporal: hour, day of week, month, weekend flag, holiday flag
- Lagged occupancy: $t - 1$, $t - 24$
- Rolling occupancy means: 3-hour, 24-hour
- Spatial: latitude, longitude, total docks
- Weather: temperature, humidity, precipitation, wind speed

Targets are multi-dimensional:

$$\mathbf{y}_t = [\text{avail_bikes}_t, \text{avail_docks}_t, \text{occupancy}_t]$$

4 Experimental Setup

4.1 Data

We use a processed station-status dataset derived from Open Bus Project feeds (2015–2019), containing over 23 million records. Each entry includes station metadata, availability counts, occupancy, and weather attributes.

Train/Test Split:

- Training: March 2015 – July 2016
- Testing: June 2017 – April 2019

A data gap necessitated a non-contiguous split.

4.2 Evaluation Metrics

Models are evaluated using:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum (y - \hat{y})^2}, \quad \text{MAE} = \frac{1}{N} \sum |y - \hat{y}|$$

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}, \quad \text{MAPE} = \frac{1}{N} \sum \left| \frac{y - \hat{y}}{y} \right|$$

4.3 Comparison Methods

We compare:

- ARIMA / ARIMAX (statistical baselines)
- LSTM (deep temporal model)
- XGBoost (tree-based ensemble)

These methods represent increasing model complexity and expressiveness.

The experimental evaluation is designed to answer the following questions: (i) how accurately can station-level availability and occupancy be forecasted, (ii) how do classical statistical models compare to modern machine learning methods, and (iii) what trade-offs exist between temporal smoothness and responsiveness to abrupt congestion events.

5 Results

This section presents the empirical performance of all evaluated models on the held-out test set. Results are reported separately for available bikes, available docks, and occupancy in order to highlight differences in model behavior across operational targets. We first compare quantitative error metrics and then provide qualitative analysis using visual diagnostics to better understand temporal tracking and prediction accuracy. The following three tables summarize the forecasting performance of each model across different evaluation metrics for the respective target variables.

Table 1: Available Bikes Forecasting Performance

Model	RMSE	MAE	R^2	MAPE (%)
XGBoost	2.09	1.18	0.957	23.19
LSTM	3.75	2.10	0.862	39.13
ARIMA	2.62	2.19	0.012	25.53
ARIMAX	2.57	2.08	0.046	24.58

Table 2: Available Docks Forecasting Performance

Model	RMSE	MAE	R^2	MAPE (%)
XGBoost	3.09	1.81	0.940	22.46
LSTM	4.31	2.49	0.884	31.90
ARIMA	2.52	1.96	0.018	11.87
ARIMAX	2.68	2.13	-0.116	12.22

Table 3: Occupancy Forecasting Performance

Model	RMSE	MAE	R^2	MAPE (%)
XGBoost	0.061	0.037	0.959	22.76
LSTM	0.111	0.065	0.867	38.94
ARIMA	0.066	0.052	0.101	15.94
ARIMAX	0.066	0.052	0.115	15.89

To complement the quantitative metrics reported above, we analyze model behavior using aggregate temporal plots and predicted-versus-actual scatter visualizations. These figures provide insight into temporal tracking ability, calibration, and systematic bias across different modeling approaches.

XGBoost achieves the strongest performance across all targets, with the highest R^2 and lowest RMSE. LSTM captures smooth temporal trends but exhibits higher variance during abrupt congestion events. ARIMA-based models show limited explanatory power due to linear assumptions.

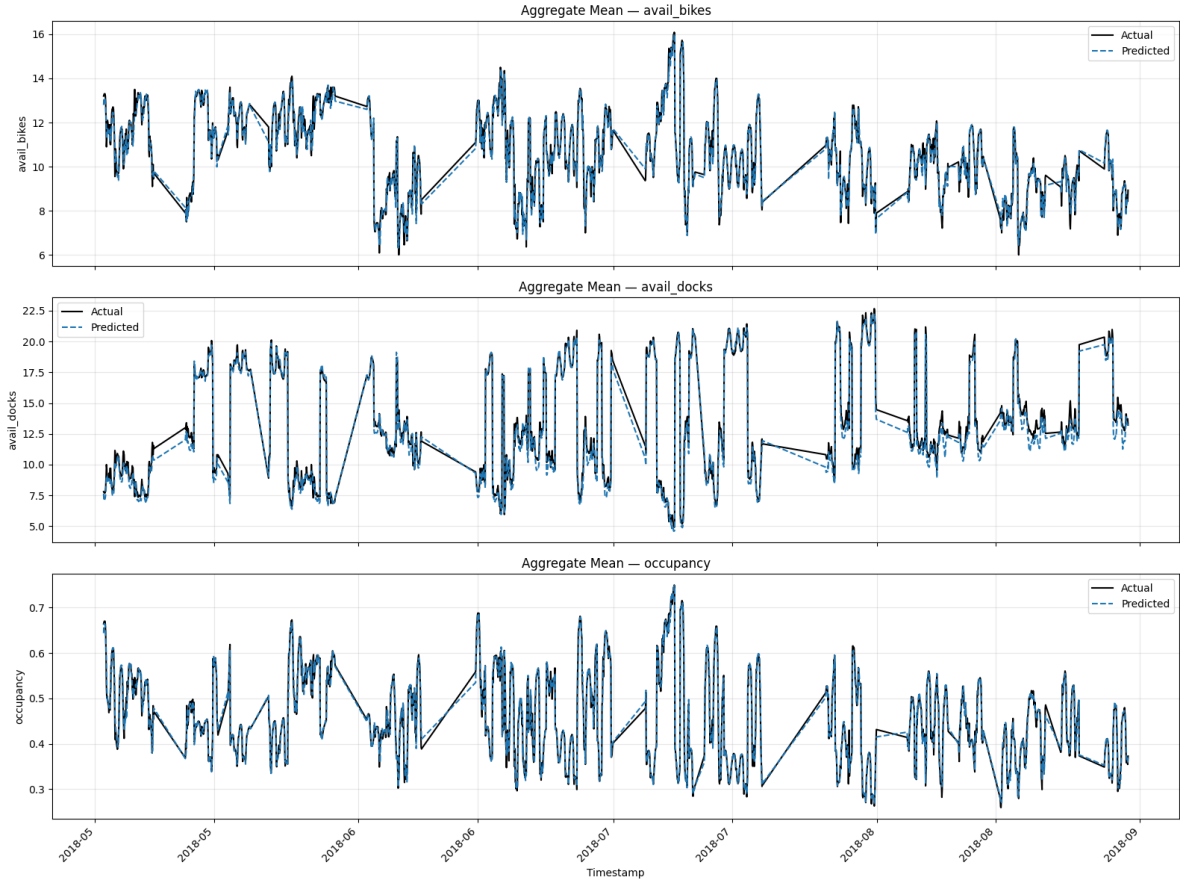


Figure 1: XGBoost aggregate mean forecasts vs. actuals.

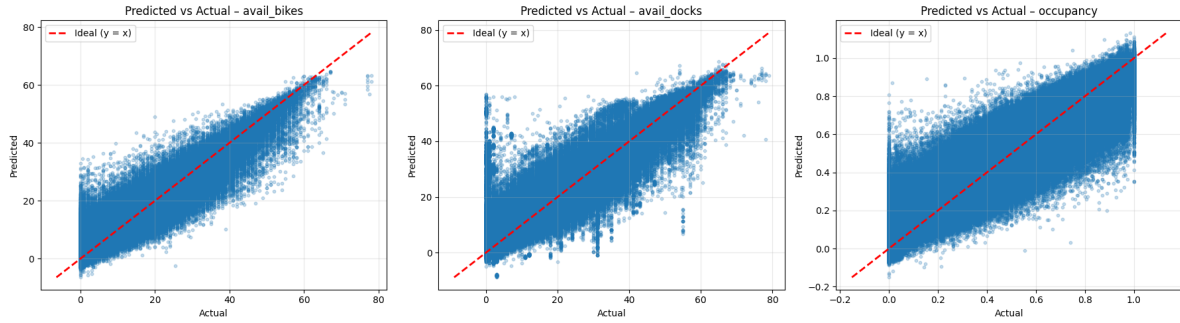


Figure 2: XGBoost predicted vs. actual scatter plots.

The following figures present the same visual diagnostics for the LSTM model, allowing a direct qualitative comparison between tree-based and deep learning approaches under identical evaluation conditions.

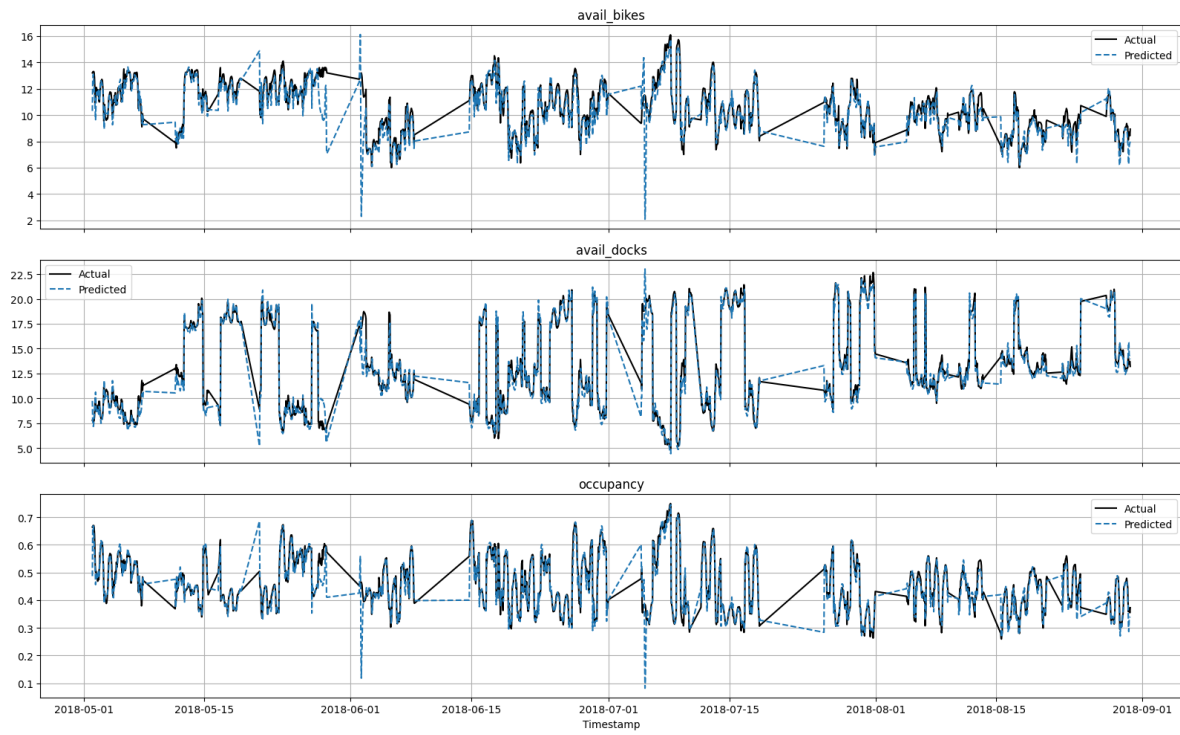


Figure 3: LSTM aggregate mean forecasts vs. actuals.

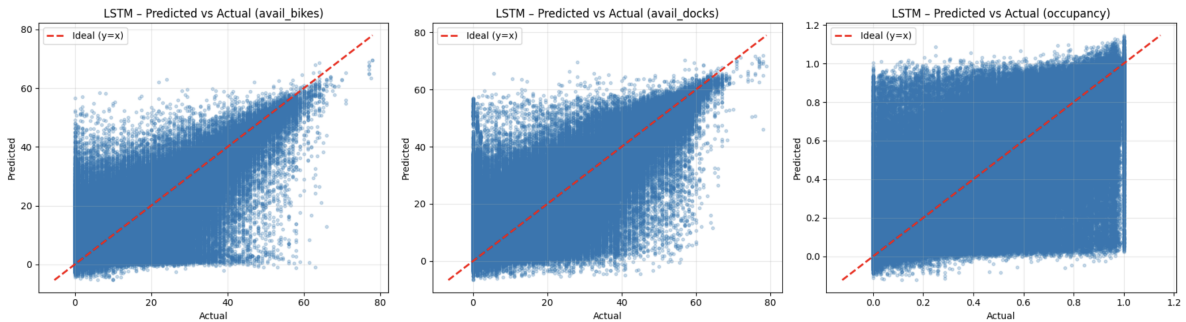


Figure 4: LSTM predicted vs. actual scatter plots.

6 Interactive Streamlit Application.

To demonstrate the practical applicability of the proposed forecasting models, we developed an interactive Streamlit-based application that operationalizes the entire congestion forecasting pipeline. The application integrates real-time Citi Bike GBFS feeds with historical station-status data to provide live monitoring, region-aware geospatial visualization, and station-level congestion risk assessment. Users can dynamically select geographic regions and individual stations, inspect current operational conditions, and compare one-step-ahead forecasts from XGBoost, LSTM, and ARIMA models against observed occupancy. The system further includes model diagnostics such as residual analysis, predicted-versus-actual visualizations, and feature importance plots to support interpretability. By combining real-time data ingestion with machine learning-driven forecasting, the application bridges the gap between offline model evaluation and actionable decision support, illustrating the feasibility of deploying the proposed methods in a real-world bike-sharing operational setting.

Interactive Deployment. We developed an interactive Streamlit application that integrates real-time Citi Bike GBFS data with historical station-status records to operationalize congestion forecasting. The system provides region-aware geospatial visualization, station-level risk assessment, and interactive comparison of XGBoost, LSTM, and ARIMA forecasts against observed occupancy. This deployment demonstrates the feasibility of translating offline machine learning models into actionable, real-time decision-support tools.

7 Conclusion

This study demonstrates that accurate forecasting of bike availability and station congestion requires direct station-status data rather than trip-level proxies. Across all evaluated targets, machine learning models substantially outperform classical time-series approaches by effectively leveraging temporal, spatial, and weather-driven signals.

Future work for this project could include:

- **Station Congestion Classification:** Extend the forecasting framework to a probabilistic classification task to estimate the likelihood of stations becoming full or empty within the next 30–60 minutes, enabling proactive congestion alerts for operators.
- **Dynamic Bike Rebalancing Optimization:** Integrate demand forecasts with optimization techniques, such as mixed-integer linear programming (MILP) or heuristic algorithms, to recommend optimal bike redistribution schedules and truck routing strategies.
- **Reinforcement Learning for Adaptive Rebalancing:** Apply reinforcement learning methods (e.g., DQN, PPO) to develop adaptive, real-time policies for bike redistribution under dynamic demand and urban traffic conditions.
- **Anomaly and Event Detection:** Incorporate anomaly detection to identify sudden demand spikes caused by special events, weather disruptions, or transit outages, improving operational responsiveness.
- **Critical Station Bottleneck Analysis:** Leverage network analytics and SHAP-based explainability to pinpoint high-impact stations and understand the key drivers of system instability.
- **Real-Time, Multi-City Deployment:** Enable real-time inference using live GBFS or station-status feeds and extend the framework to support scalable deployment across multiple cities.

References

- [1] Box, G., Jenkins, G., Reinsel, G., & Ljung, G. (2015). Time Series Analysis: Forecasting and Control.
- [2] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *KDD*.
- [3] Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*.
- [4] Taylor, S. J., & Letham, B. (2018). Forecasting at scale. *The American Statistician*.
- [5] Yao, H., et al. (2018). Modeling spatial-temporal dynamics using graph convolutional networks. *AAAI*.
- [6] Hyndman, R., & Athanasopoulos, G. (2018). Forecasting: Principles and Practice.